

O Ecossistema de Streaming: Do Motor Técnico à Plataforma Comercial

O framework de vídeo da nossa operadora de streaming atingiu sua maturidade arquitetural. Nas etapas anteriores deste projeto, você e sua equipe de engenharia resolveram os desafios estruturais de baixo nível. Inicialmente, consolidamos a construção e a ramificação da Timeline e seus componentes de renderização gráfica. No segundo momento, otimizamos o protocolo de distribuição, garantindo a injeção dinâmica de conteúdos (trailers e créditos) e resolvendo severos gargalos de memória na leitura dos bytes nativos.

Agora, a operadora de streaming está pronta para chegar ao usuário final. A diretoria da operadora aprovou o lançamento oficial do aplicativo de assinantes e do painel de administração corporativa. Isso exige que o nosso sistema expanda suas fronteiras: precisamos conectar o Domínio Técnico (como o vídeo funciona) ao Domínio Comercial (como o vídeo é vendido, agrupado e analisado).

Para que essa nova arquitetura funcione sem violar os princípios SOLID estabelecidos nas fases anteriores, a equipe de modelagem definiu uma fronteira clara de responsabilidades operando através do princípio da Composição:

O Domínio Técnico: A interface **RenderableContent** e a classe **Timeline**, que até agora eram as entidades centrais, passam a atuar estritamente como o "motor técnico" da mídia. Elas e as trilhas (**Tracks**) continuam existindo e operando, mas ignoram completamente conceitos de negócio como preços, assinaturas, catálogos e promoções.

O Domínio Comercial (A Vitrine): Introduzimos a entidade **Filme** (e futuramente suas variações, como Clipes, Séries e Episódios). O **Filme** atua como a "casca comercial" consumida pelo usuário. Ele encapsula atributos de negócio (título, preço, arte de capa, sinopse) e contém uma **Timeline** internamente. Quando o aplicativo precisa de informações referentes ao domínio técnico, como por exemplo a duração do filme, a classe **Filme** simplesmente delega essa pergunta para a sua Timeline interna. Quando o aplicativo precisa de informações do domínio comercial, como por exemplo o preço do filme, a própria classe **Filme** se encarrega de prover esta informação.

Nesta terceira e última etapa, os seus desafios arquiteturais deixarão de envolver a manipulação de bytes ou codecs físicos. As novas questões agora envolvem a orquestração de catálogos promocionais complexos, a extração de dados analíticos sem corromper as classes estruturais, a comunicação reativa entre serviços de notificação e o gerenciamento de estados dinâmicos na interface do usuário.

A evolução exige novos padrões. Analise os cenários a seguir e aplique as melhores soluções para os problemas apresentados no novo ecossistema do framework.

Questão I - Para a Black Friday, a operadora de streaming vai vender pacotes fechados de conteúdo com desconto (ex: "Trilogia Matrix", que contém 3 filmes e 1 documentário bônus). O sistema deve permitir criar "Super Pacotes" que contenham outros pacotes dentro (ex: "Coleção Sci-Fi", que contém o pacote "Trilogia Matrix", o pacote "Star Wars", a série "Black Mirror" e um filme avulso "Blade Runner").

O carrinho de compras da aplicação precisa calcular o preço total e a duração dos produtos de forma elegante, extensível e transparente, tratando filmes avulsos e pacotes complexos (aninhados) exatamente da mesma maneira. Além disso, a primeira versão do processo de instanciar esses pacotes aninhados resultou em construtores gigantescos, repetitivos e ilegíveis no código (ex: **new**

Pacote("SciFi", new Pacote("Matrix", new Filme(), new Filme()), new Serie("BlackMirror", new Episodio(), new Episodio(), new Episodio()) new Filme())), o que está tornando a criação de novas promoções uma atividade propícia a erros e difícil de manter. Proponh e implemente uma solução que combine padrões de projeto, que enderece os problemas apresentados.

Questão II - Uma funcionalidade muito apreciada pelos clientes na aplicação é a possibilidade de criar playlists personalizadas. Estas playlists podem combinar tantos os diversos tipos disponibilizados no catálogo de vendas da operadora como outros elementos sem relação, como os MP3's e pequenos vídeos carregados pelo próprio cliente.

A equipe de Ciência de Dados solicita frequentemente novas operações analíticas sobre essas playlists. Já foi solicitada uma operação para calcular a "Largura de Banda Total" gasta por playlist. Em outro momento, um extrator de "Relatório com o Nome dos Elementos da Playlist". Agora chegou um pedido de "Exportador para XML". Estas operações resultaram em mudanças nas classes MP3, VideoClique, Episodio e Filme. A cada nova requisição da equipe de dados é necessário modificar as classes originais, o que já gerou a injeção de erros como efeitos colaterais, quebrando o princípio do Aberto/Fechado e da Responsabilidade Única. Proponha e implemente uma solução baseada em padrões de projetos, que enderece os problemas apresentados.

Normas de Entrega

1. As soluções devem ser escritas em código JAVA
2. O trabalho deverá ser feito em equipe de até três pessoas e preferencialmente de três pessoas.
3. A entrega deve ser feita mediante a disponibilização do projeto em ambiente git (github) com acesso compartilhado para o usuário pooinf008
4. A data do último commit NÃO pode ser posterior ao dia 19/07/2026 às 23:59
5. Somente será avaliada a branch principal
6. O link do projeto no github deve ser remetido para o endereço eletrônico poo.inf008@gmail.com
7. Este mail deverá conter no assunto (subject): [inf0112026.1aval3] - O assunto será usado para filtragem automática, portanto atentem-se ao formato indicado.
8. Os padrões de projeto aplicados devem ser detalhados no readme.md, incluindo: O nome do padrão de projeto. Uma breve justificativa para sua escolha. A identificação clara de cada classe do projeto e qual papel (participante) ela assume no padrão (ex: Memento, Originator, Caretaker)